

LISTES AVEC SENTINELLE

Idée :

Ajout d'une cellule sentinelle en tête de liste

- Suivant (tête) : pointe sur la première cellule réelle de la liste.
- Le membre donné de la première cellule n'est pas utilisé.
- Toutes les fonctions prennent comme adresse de liste, l'adresse de la sentinelle.



Les mises à jour sur la liste, ne modifie pas l'adresse de la liste.

Parmi les défauts des listes que nous avons présentées jusqu'à présent, on trouve principalement :

- Les difficultés liées à la manipulation du pointeur tête : insertion et suppression en tête (→ modification de l'adresse de la liste).

Liste circulaire avec sentinelle

- Les difficultés liées au cas particulier de la liste vide.
- Le fait qu'on soit obligé, quand on parcourt la liste à l'aide d'un pointeur l, de tester continuellement si la liste est vide, alors qu'une seule cellule de la liste a son membre suivant à NULL.

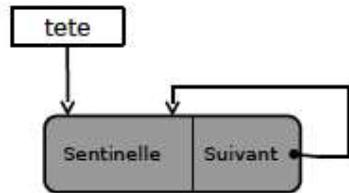
Structure de données

- listeCS  Liste

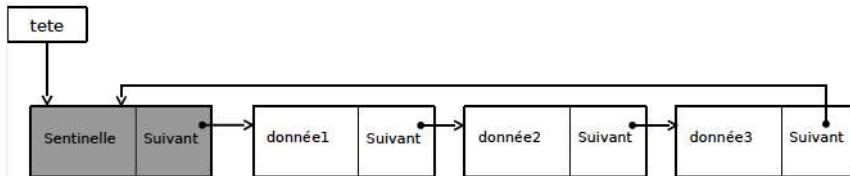
Définition

Le dernier élément pointe sur le premier élément de la liste.

Liste circulaire vide



Liste circulaire avec 3 éléments



Avantages

- Simplifie la gestion des cas limites : cas où la liste est vide (la liste contient toujours au moins une cellule)
- Cela évite les problèmes de tests avec la valeur NULL

1) Une liste circulaire avec sentinelle contient toujours au moins un élément. Cet élément est toujours le premier élément de la liste et ne contient pas **de donnée significative**.

Cela signifie que la donnée qui y est stockée ne doit pas être considérée comme faisant partie de ce qui est stocké dans la liste.

2) Comme son nom l'indique, une liste circulaire avec sentinelle est circulaire, c'est-à-dire que le membre suivant de la dernière cellule contient la même donnée que la liste L, à savoir l'adresse de la première cellule (à savoir, la donnée de la sentinelle).

Dans une telle liste, il convient donc de distinguer la première cellule (physique) de la liste, qui est la cellule sentinelle, et la première cellule significative, qui est celui qui suit la sentinelle (s'il y en a un).

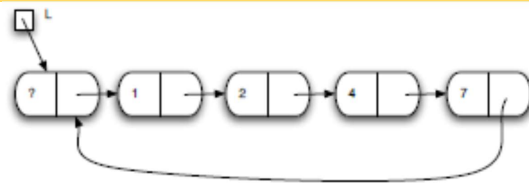


FIG. 5.5 – Un exemple de liste circulaire avec élément pré-tête.

**En quoi ces deux différences résolvent-elles nos problèmes ?
Passons-les en revue :**

- Il n'y a plus vraiment d'insertion et de suppression en tête, puisque le premier élément significatif d'une liste circulaire avec sentinelle est en fait le second de la liste.

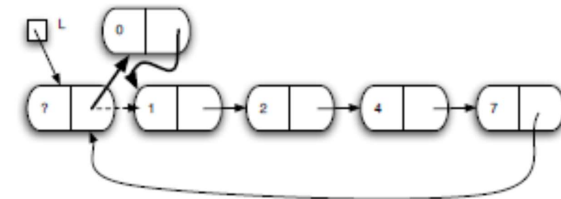


FIG. 5.6 – Un exemple d'insertion en tête dans une liste circulaire avec élément pré-tête.

- La seule modification de la liste L aura donc lieu au moment de la création de la liste.

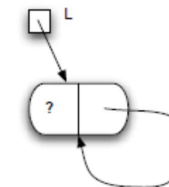


FIG. 5.7 – Une liste circulaire avec élément pré-tête vide.

- Une liste circulaire avec sentinelle, même vide, contient toujours au moins une cellule (la cellule sentinelle). On n'aura donc jamais de pointeur L égal à NULL.

Principe de l'algorithme de recherche

La recherche d'une valeur v dans la liste sera également simplifiée.

En effet, comme la valeur contenue dans l'élément sentinelle n'est pas significative, rien ne nous empêche de la modifier.

Ainsi, si on recherche la valeur dans la liste, on peut placer cette valeur dans l'élément sentinelle. On commence la recherche de v sur le premier élément significatif. Comme la liste est circulaire, on est certain de trouver l'élément v :

- Soit parmi les éléments significatifs,
- Soit dans l'élément sentinelle, qu'on rencontrera immanquablement comme successeur du dernier élément.

Ces deux cas peuvent aisément être discriminés : tout élément significatif a une adresse $\neq L$, et l'élément sentinelle est pointé par L .

Le seul test à faire en parcourant une telle liste à l'aide du pointeur l est donc $donnee(l) \neq v$. La Fig. 5.8 illustre cela.

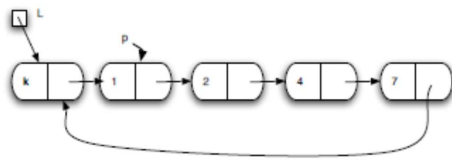


FIG. 5.8 – La recherche de l'information v dans une liste circulaire avec sentinelle.

De manière générale, ces listes ont l'avantage de ne pas utiliser de pointeurs NULL, ce qui limite les risques d'erreur.

Algorithmes

Initialisation à vide de la Liste Circulaire

Fonction `initListeLCS : {} → ListeCS`

La fonction `initListeCS` renvoie une liste circulaire contenant uniquement la sentinelle :

Fonction `initListeCS(sentinelle) → ListeCS`

P.F sentinelle : `TElement (E)`

Début

`Cel ← allocMem()`

`Cel.donnee ← sentinelle`

`Cel.suivant ← cel`

`renvoyer cel`

FIN

Teste si une liste est vide :

Fonction estVideLCS : ListeCS → Booléenne

La fonction estVideCS est une fonction booléenne qui reçoit une liste en paramètre, et qui renvoie vrai si et seulement la liste est vide.

C'est le cas si et seulement si l'élément sentinelle à son membre suivant qui pointe vers lui-même.

Fonction estVideLCS(lcs) → Booléenne

P.F lcs : ListeCS (E)

Début

Renvoyer suivant(lcs) = lcs

Fin

Première cellule de la liste

Precondition : non estVideLCS(l)

Fonction teteCS : ListeCS → ListeCS

La fonction tête reçoit une liste et renvoie un pointeur vers la première cellule de la liste.

Il retourne la première cellule significative pointée par **suivant(lcs)**, s'il existe (c'est-à-dire si la liste n'est pas vide).

Fonction teteLCS(lcs) → ListeCS

P.F lcs : ListeCS (E)

Début

Renvoyer suivant(lcs)

FIN

Cette fonction ne réalise que des tests et des traitements en temps constant.

Dernière cellule de la liste

Precondition : non estVide(l)

Fonction queueLCS : ListeCS → ListeCS

La fonction queue reçoit une liste et renvoie un pointeur vers la dernière cellule de la liste.

On parcourt la liste jusqu'à atteindre la dernière cellule. C'est-à-dire attendre la cellule dont le membre suivant est L.

Comme il est nécessaire de parcourir toute la liste, cette fonction est en $O(n)$, où n est le nombre de cellules de la liste.

Fonction QueueLCS(lcs) → ListeCS

P.F lcs : ListeCS (E)

Début

ll ← lcs

Tantque suivant(ll) ≠ lcs faire

ll ← suivant(ll)

FinTq

Renvoyer ll

Fin

Insertion d'un élément à la k^{ème} position

Fonction inserKPos : Entier x TElement x ListeCS → ListeCS

Cette fonction reçoit une liste L, un entier k et un élément elt. Elle insère l'élément elt dans la liste, à la k^{ème} position.

Cette fonction profite grandement de l'élément sentinelle, ce qui évite de traiter l'insertion en tête comme un cas particulier.

Remarquons que si la liste ne contient pas assez d'éléments (moins de k-1), l'insertion n'est pas possible.

Principe :

1) On détermine l'adresse d'insertion s'elle existe.

2) On réalise l'insertion (càd les chaînages)

Procédure inserKPos(k, elt, l)

P.F k : Entier (E)

elt : TElement (E)

lcs : ListeCS (E)

Début

akCel ← adrKPosition(k, lcs) //adresse kéme cellule

si non estVideLCS(akCel) alors

cel ← allocMem()

cel.donnee ← elt

cel.suivant ← suivant(akCel)

akCel.suivant ← cel

Fsi

FIN

Recherche l'adresse de la Kéme Cellule

Fonction adrKPosition : Entier x Liste → Liste

On parcourt la liste à l'aide du pointeur ll, en commençant sur le premier élément significatif. À chaque nouvel élément rencontré, on incrémente un compteur. La boucle s'arrête soit parce que le compteur vaut 1, soit parce que ll pointe vers la sentinelle.

Fonction adrKPosition(k, lcs) → ListeCS

P.F k : entier (E)

lcs : ListeCS (E)

Début

ll ← suivant(lcs)

tantque k>1 et suivant(ll) ≠ lcs faire

k ← k-1

ll ← suivant(ll)

Ftq

Si k=1 alors

Renvoyer ll

Sinon

Renvoyer NULL

Fsi

FIN

Recherche d'un élément

Fonction rechElt : TElement x Liste → Liste

La fonction Recherche reçoit un pointeur vers la tête d'une liste, et un élément « elt ».

Elle renvoie un pointeur vers le premier élément qui contient l'élément « elt », ou NULL si aucune cellule ne contient cet élément.

Principe

On stocke l'élément recherché dans l'élément sentinelle. Ceci permet de diminuer le nombre de tests à effectuer dans la boucle tant que qui parcourt la liste. Le seul test nécessaire est **donnee(II) <> elt**.

A la sortie de la boucle, on a donc la garantie que **II** pointe vers une cellule dont le membre donné contient élément recherché. Il suffit de tester si **II** est égal à **Lcs**, afin de déterminer si l'élément trouvé est un élément significatif de la liste, ou bien l'élément sentinelle, auquel cas on est sûr que la liste ne contient pas d'élément dont la donnée est **l'élément recherché**.

Fonction rechElt(elt, l) → ListeCS

P.F

Elt : TElement (E)

lcs : ListeCS (E)

Début

II ← tete(lcs)

lcs.donnee ← elt

tant que donnee(II)≠elt faire

II ← suivant(II)

Ftq

Si II=lcs alors

renvoyer NULL

Si non

renvoyer II

Fsi

Fin

Insertion triée dans une liste circulaire avec sentinelle

Le principe est simple :

On détermine d'abord l'adresse d'insertion

Puis on effectue l'insertion par modification de chaînage.

Recherche de l'adresse d'insertion

En mettant une donnée VMAX à la place de la sentinelle, la recherche de l'adresse d'insertion est simplifiée.

Fonction $\text{adrInser}(\text{elt}, \text{l}) \rightarrow \text{ListeCS}$

P.F $\text{elt} : \text{TElement} (E)$

$\text{lcs} : \text{ListeCS} (E)$

Début

$\text{lcs.donnee} \leftarrow \text{VMAX}$

$\text{ll} \leftarrow \text{lcs}$

 tantque $\text{elt} > \text{donnee}(\text{suivant}(\text{ll}))$ faire

$\text{ll} \leftarrow \text{suivant}(\text{ll})$

 ftq

 renvoyer ll

Fin

Procédure $\text{inserTriéLCS}(\text{elt}, \text{lcs})$

P.F $\text{elt} : \text{TElement} (E)$

$\text{lcs} : \text{ListeCS} (E)$

Début

$\text{ll} \leftarrow \text{adrInser}(\text{elt}, \text{lcs})$

$\text{cel} \leftarrow \text{allocMem}()$

$\text{cel.donnee} \leftarrow \text{elt}$

$\text{cel.suivant} \leftarrow \text{suivant}(\text{ll})$

$\text{ll.suivant} \leftarrow \text{cel}$

Fin

Algorithmes de Suppression

Suppression par position

Suppression par valeur dans une liste quelconque

Suppression par valeur dans une liste triée

Suppression d'une liste